

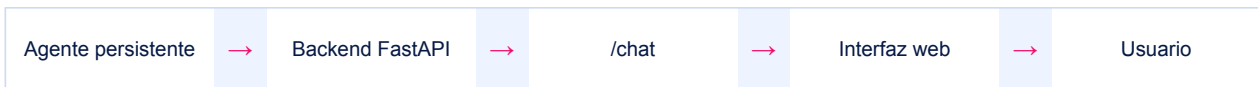
Sesión 3 · De agente a aplicación

Persistencia, chat, FastAPI, interfaz web y despliegue seguro.

03

Tu meta de aprendizaje

Reutilizar un agente persistente, mantener memoria por conversación, exponerlo mediante FastAPI y preparar un despliegue seguro.



1. Publicar no significa recrear

En el SDK 2.x el agente se identifica por nombre y tiene versiones. La app apunta al AGENT_NAME; crear una versión nueva es una operación de publicación, no algo que se haga por petición.

```
openai = project.get_openai_client(agent_name=os.environ["AGENT_NAME"])
```

REGLA

Publica una versión cuando cambie la definición; conversa muchas veces usando el mismo nombre.

2. Crea el agente definitivo

- 1 Configura
Activa .env, ejecuta az login y revisa .env.
- 2 Publica
Ejecuta crear_agente.py para crear una versión.
- 3 Guarda
Añade AGENT_NAME al .env, nunca secretos.
- 4 Verifica
Abre un OpenAI client vinculado al nombre del agente.

```

FOUNDRY_PROJECT_ENDPOINT=https://.../api/projects/tu-proyecto
FOUNDRY_MODEL_NAME=tu-deployment
AGENT_NAME=agente-publicado
  
```

3. Construye un chat con memoria

La memoria conversacional depende de reutilizar el mismo conversation_id. Una conversación nueva equivale a una sesión nueva.

```

conversation = openai.conversations.create()
while True:
    texto = input("Tú: ")
    if texto.lower() in {"salir", "exit", "quit"}:
        break
    response = openai.responses.create(
        conversation=conversation.id,
        input=texto,
    )
    print(response.output_text)
  
```

4. Expón /chat con FastAPI

```
pip install fastapi uvicorn
uvicorn api:app --reload --port 8000
```

Contrato de entrada: mensaje obligatorio y conversation_id opcional. Contrato de salida: respuesta y conversation_id. El cliente conserva el ID para mantener contexto.

```
class Entrada(BaseModel):
    mensaje: str
    conversation_id: Optional[str] = None

@app.post("/chat")
def chat(body: Entrada):
    cid = body.conversation_id
    if not cid:
        cid = openai.conversations.create().id
    response = openai.responses.create(
        conversation=cid, input=body.mensaje
    )
    return {"respuesta": response.output_text,
            "conversation_id": cid}
```

CORS

allow_origins=["*"] es aceptable solo en la práctica local. En producción limita el origen al dominio real.

5. Prueba por capas

1

Backend
Abre /docs, envía un mensaje y conserva conversation_id.

2

Memoria
Envía una segunda petición con ese ID y verifica contexto.

3

Frontend
Abre index.html con la API local activa.

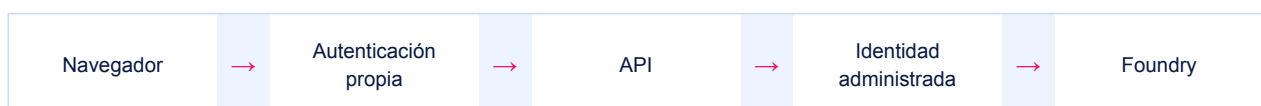
4

Falla controlada
Prueba mensaje vacío, AGENT_NAME incorrecto y backend detenido.

5

Observación
Registra tiempo, errores y herramientas sin exponer datos sensibles.

6. Arquitectura de producción



- Autenticación del usuario: protege quién puede llamar /chat.
- Identidad administrada: autentica el servidor ante Azure sin claves.
- Persistencia: almacena conversation_id asociado al usuario, con expiración y privacidad.
- Límites: rate limit, tamaño máximo, timeout y control de costos.

- Observabilidad: correlación, latencia y errores de Responses; evita registrar PII innecesaria.

7. Despliega opcionalmente en Container Apps

```
az containerapp up \
  --name agente-api \
  --resource-group TU_GRUPO \
  --location TU_REGION \
  --source . \
  --ingress external \
  --target-port 8000

az containerapp identity assign \
  --name agente-api \
  --resource-group TU_GRUPO \
  --system-assigned
```

Después asigna a esa identidad el rol mínimo requerido sobre el recurso/proyecto de Foundry. Configura variables o secretos en Container Apps y restringe el acceso público antes de compartir la URL.

8. Checklist de salida

- AGENT_NAME apunta al agente publicado.
- Dos mensajes con el mismo conversation_id conservan contexto.
- /docs responde y muestra un error controlado si Foundry falla.
- CORS, autenticación y límites están definidos para producción.
- No hay .env ni credenciales en Git.
- Existe una instrucción de limpieza y control de costos.

9. Diagnóstico rápido

KeyError AGENT_NAME. Añádalo a .env y confirma load_dotenv().

Agent not found. Nombre y endpoint deben pertenecer al mismo proyecto.

CORS / fetch. Comprueba origen, puerto y que uvicorn siga activo.

401 en la nube. Activa identidad administrada y asigna el rol adecuado.

La app no escucha. En nube usa host 0.0.0.0 y el puerto configurado.